SNOWFLAKE

Architecture · UI · Warehouses · Data Types · Tables · Cloning · Time Travel · Views · Stages · Loading

SESSION 1 — Introduction & Architecture

1.1 What is Snowflake?

Snowflake is a cloud-native data platform delivered as a Software-as-a-Service (SaaS). Unlike traditional databases you install on a server, Snowflake runs entirely on cloud infrastructure (AWS, Azure, or GCP) and you access it through a browser or SQL client.

Think of it like this analogy:

- Traditional database = owning a car (you buy it, maintain it, fix it when it breaks)
- Snowflake = using Uber (you just ride, someone else handles the engine)

Key characteristics of Snowflake:

- No hardware to manage — fully managed by Snowflake
- Compute and storage are SEPARATED — scale independently
- Pay only for what you use (per second billing)
- Supports structured data (tables) AND semi-structured data (JSON, XML, Parquet, Avro, ORC)
- Multi-cloud: runs on AWS, Azure, and Google Cloud
- Standard SQL — if you know SQL, you already know most of Snowflake

1.2 Why Was Snowflake Created? — The Problem with Old Architectures

Architecture #1 — Shared Disk Architecture (Traditional Databases)

In a shared disk system, ALL compute nodes (servers doing the work) share ONE central storage. Examples: Oracle RAC, older MPP databases.

How it works:

- Multiple CPU servers are connected to a single large storage pool
- All servers read/write from the same disk
- If one server is busy, others wait

Real-world example:

Imagine 10 chefs (compute nodes) all working in the SAME kitchen (shared disk). If Chef A needs the knife that Chef B is using, Chef A must wait. The kitchen becomes a bottleneck.

Problems:

- Storage bottleneck: all nodes compete for I/O access
- Hard to scale out — adding more nodes doesn't help much if storage is the bottleneck
- Single point of failure on storage side

Architecture #2 — Shared Nothing Architecture (Traditional MPP like Redshift)

In a shared nothing system, EACH compute node has its OWN dedicated storage. Amazon Redshift, Teradata, Greenplum use this approach.

How it works:

- Each node has CPU + its own local disk
- Data is partitioned (split) across nodes

- No sharing of resources between nodes

Real-world example:

Now imagine 10 chefs each with their OWN kitchen. No waiting for shared tools. BUT if you want to hire 5 more chefs, you must build 5 more complete kitchens (storage + compute together). That is expensive and inflexible.

Problems:

- Scaling requires buying MORE storage even if you only need more compute
- Data skew: if some nodes have more data than others, they become bottlenecks
- Cluster resizing is painful and often requires downtime
- Cannot scale compute independently from storage

Architecture #3 — Snowflake's Multi-Cluster Shared Data Architecture

Snowflake invented a new architecture that separates storage completely from compute. This is the breakthrough that makes Snowflake special.

How it works:

- Storage layer: ALL data lives in cloud object storage (S3/Azure Blob/GCS)
- Compute layer: Virtual Warehouses (clusters of VMs) process queries
- The two layers talk through a high-speed network
- Multiple warehouses can access the SAME data simultaneously

Real-world example:

Think of Netflix. The movies (data) are stored in one central library (S3). Any TV in the world (compute) can stream them independently. You do not need a TV in the library room. The TV and the library are separate things.

Benefits:

- Scale compute without scaling storage (and vice versa)
- Multiple teams can run queries on same data at the same time without interfering with each other
- Pay for storage and compute separately
- Resume/pause compute in seconds

Feature	Shared Disk	Shared Nothing	Snowflake
Storage Location	Centralized shared disk	Each node has own disk	Cloud object storage (S3 etc.)
Compute	Multiple CPUs	Each node = CPU+Disk	Virtual Warehouses (independent)
Scalability	Limited	Hard to separate	Independent scale of compute & storage
Cost	Fixed hardware	Fixed hardware	Pay per use
Examples	Oracle RAC	Redshift, Teradata	Snowflake

1.3 Snowflake 3-Layer Architecture — Deep Dive

Snowflake's architecture has THREE distinct layers. Understanding this is fundamental to everything else in this course.

LAYER 1 — Database Storage Layer

This is where ALL your data lives permanently.

- Uses cloud object storage: AWS S3, Azure Blob Storage, or Google Cloud Storage
- Data is stored in a proprietary compressed columnar format (not raw files you can directly access)
- Snowflake AUTOMATICALLY handles: compression, encryption, metadata, statistics, and micro-partitioning
- You never interact with this layer directly — Snowflake manages it for you
- Data persists here even when no compute (warehouse) is running

Key point: You pay for storage 24x7 whether your warehouse is running or not. Storage is cheap (~$23/TB/month on AWS). Compute is where most costs come from.

LAYER 2 — Query Processing Layer (Virtual Warehouses)

This is where all the actual computation (running queries, loading data, transformations) happens.

- Made up of Virtual Warehouses — clusters of virtual machines in the cloud
- Each Virtual Warehouse is a set of compute nodes working together
- Warehouses can be STARTED and STOPPED on demand — in seconds
- When stopped, you pay ZERO for compute
- Multiple warehouses can run simultaneously on the SAME data

Example: Your BI team runs their dashboards on Warehouse_BI. Your data engineers run ETL on Warehouse_ETL. Both access the same tables — no conflicts, no performance impact on each other.

LAYER 3 — Cloud Services Layer

This is the 'brain' of Snowflake. It coordinates everything and runs continuously in the background.

What it manages:

- Authentication & Authorization: Who can log in? What can they see?
- Query compilation & optimization: Rewrites your SQL to run as fast as possible
- Metadata management: Keeps track of all tables, columns, schemas, statistics
- Query parsing: Understands your SQL before sending it to compute
- Infrastructure management: Manages the actual cloud VMs for warehouses
- Transaction management: Ensures ACID compliance

Important pricing note: Cloud Services are mostly free — you are only billed if Cloud Services usage exceeds 10% of your daily compute credits (very rare in normal use).

> **TIP** Memory trick for the 3 layers: Think of a restaurant. Storage = the pantry (food stored). Compute = the kitchen (where cooking happens). Cloud Services = the restaurant manager (coordinates everything, ensures quality).

1.4 Snowflake Editions

Snowflake is available in 4 editions with different features and price points. The free trial gives you Standard Edition with $400 worth of credits.

Edition	Best For	Key Features	Limitations
Standard	Dev/Test, small teams	All core SQL, basic data sharing, encryption at rest	No multi-cluster warehouses, limited security features
Enterprise	Production workloads	Multi-cluster WH, column-level security, 90-day time travel	Costs more
Business Critical	Finance, Healthcare, Gov	HIPAA/PCI compliance, private link, higher security	Most expensive
Virtual Private	Defense, ultra-secure	Dedicated VPC, no shared infrastructure	Very high cost, restricted features

 FREE TIER Free trial = Standard Edition with $400 credits. Most labs in this course WORK on free tier. Where features are not available (e.g., multi-cluster warehouses require Enterprise), a NOTE will clearly indicate this.

SESSION 2 — Snowflake UI & Virtual Warehouses

2.1 Snowflake UI Walkthrough (Snowsight)

Snowflake's modern web UI is called Snowsight. Let us explore every section.

How to access: Log in at app.snowflake.com

Left Navigation Panel — Key Areas:

- Worksheets: Write and run SQL queries. Think of these as your SQL notebook.
- Databases: Browse all databases, schemas, tables, views you have access to.
- Warehouses: Create, manage, and monitor your compute clusters.
- Data: Data marketplace, data sharing, private exchanges.
- Activity: Query history, copy history, resource monitors.
- Admin: User management, roles, account settings.

SnowSQL — Command Line Client

SnowSQL is Snowflake's command-line tool. It lets you run SQL from your terminal, which is useful for scripting and automation.

Installation steps:

1. Go to: https://docs.snowflake.com/en/user-guide/snowsql-install-config.html
2. Download the installer for your OS (Windows .msi, macOS .pkg, Linux .bash)
3. Run the installer
4. Open your terminal and type: snowsql --version (should print version number)
5. Connect to Snowflake:

```
snowsql -a <account_identifier> -u <your_username>
-- Example:
snowsql -a xy12345.us-east-1 -u john_doe
-- It will prompt for your password
```

 NOTE Your account identifier is found in Settings > Account. It looks like: ab12345.us-east-1.aws

 TIP For this course, the Snowsight web UI is sufficient. SnowSQL is optional but good to know for automation.

2.2 Virtual Warehouses — The Compute Engine

A Virtual Warehouse (VW) is a named cluster of computing resources used to execute SQL queries and DML operations. Every query in Snowflake needs a running warehouse.

Warehouse Sizes

Warehouses come in T-shirt sizes. Each size doubles the compute power AND the cost:

Size	Credits/Hour	Nodes	Use Case
X-Small (XS)	1	1 node	Dev, testing, small queries
Small (S)	2	2 nodes	Light production queries
Medium (M)	4	4 nodes	Standard production
Large (L)	8	8 nodes	Heavy queries, large datasets

X-Large (XL)	16	16 nodes	Very large datasets
2X-Large	32	32 nodes	Data science, ML workloads
3X-Large	64	64 nodes	Massive batch processing
4X-Large	128	128 nodes	Largest workloads

 TIP Always start with XS or S. You can easily resize up. Using a 4XL for a simple SELECT is like using a bulldozer to plant a flower.

Scaling Concepts: Scale Up vs Scale Out

Scale Up (Resize)

Changing a warehouse from a smaller size to a larger size. Use this when a SINGLE query is slow because it needs more memory or CPU.

Example:

- Your ML team runs a complex join across 500 million rows. On Medium it takes 10 minutes.
- You scale up to X-Large. Now the same query takes 3 minutes.
- Resize back to Medium after the job completes to save credits.

```
-- Scale Up:
ALTER WAREHOUSE my_wh SET WAREHOUSE_SIZE = 'XLARGE';

-- Scale back down:
ALTER WAREHOUSE my_wh SET WAREHOUSE_SIZE = 'MEDIUM';
```

Scale Out (Multi-Cluster — Enterprise Feature)

Adding MORE clusters of the same size to handle MORE concurrent users. Use this when MANY users run queries simultaneously and have to wait.

Example:

- 100 analysts all run reports at 9 AM. With 1 cluster (single-cluster WH), queries queue up.
- Enable multi-cluster: min 1, max 5 clusters.
- Snowflake automatically starts cluster 2 when queue builds up, then cluster 3, etc.
- When load drops, extra clusters shut down automatically.

```
-- Create multi-cluster warehouse (Enterprise+):
CREATE WAREHOUSE enterprise_wh
  WAREHOUSE_SIZE = 'MEDIUM'
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 5
  SCALING_POLICY = 'STANDARD';  -- or ECONOMY
```

 FREE TIER Multi-cluster warehouses require Enterprise Edition. On Standard/Free tier, MAX_CLUSTER_COUNT can only be 1.

Scale Down

Reducing warehouse size or suspending it when not needed. This is how you save money.

```
-- Suspend warehouse (stop billing for compute):
ALTER WAREHOUSE my_wh SUSPEND;
```

```
-- Resume when needed:
ALTER WAREHOUSE my_wh RESUME;

-- Auto-suspend after 5 minutes of inactivity:
ALTER WAREHOUSE my_wh SET AUTO_SUSPEND = 300;  -- 300 seconds

-- Auto-resume when a query is submitted:
ALTER WAREHOUSE my_wh SET AUTO_RESUME = TRUE;
```

 TIP ALWAYS set AUTO_SUSPEND on your warehouses. Forgetting to do this is the #1 cause of unexpected Snowflake bills.

Scaling Policies (for Multi-Cluster Warehouses)

These control how aggressively Snowflake starts and stops additional clusters:

Policy	Behavior	Use Case
STANDARD	Starts additional cluster immediately when any query is queued. Shuts down after 2-3 minutes of idle.	When query latency/speed matters most. Busy BI dashboards.
ECONOMY	Waits until queuing will save 6 minutes worth of credit before starting new cluster. Shuts down faster.	When cost savings matter more than speed. Batch ETL with flexible SLAs.

 FREE TIER Scaling policies only apply to multi-cluster warehouses (Enterprise+). On free tier, only one cluster runs at a time.

2.3 Creating a Warehouse — Lab 1

 NOTE This lab works on FREE tier Snowflake.

Steps:

6. Log in to Snowsight (app.snowflake.com)
7. Click 'Warehouses' in the left panel
8. Click '+ Warehouse' button
9. Fill in the details as below and click Create

```
-- Lab 1: Create your first Virtual Warehouse
CREATE WAREHOUSE training_wh
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 120         -- suspend after 2 min idle
  AUTO_RESUME = TRUE
  COMMENT = 'Training warehouse for labs';

-- Verify it was created:
SHOW WAREHOUSES;

-- Use it:
USE WAREHOUSE training_wh;

-- Check its status:
```

```
SELECT SYSTEM$WAREHOUSE_STATUS('training_wh');

-- Suspend it to save credits:
ALTER WAREHOUSE training_wh SUSPEND;
```

Questions — Session 1 & 2

Q1	What is the key difference between Shared Disk and Shared Nothing architecture?
Q2	Why did Snowflake separate compute from storage? What problems does this solve?
Q3	Name and explain the 3 layers of Snowflake architecture.
Q4	What is the difference between Scale Up and Scale Out in Snowflake?
Q5	What does AUTO_SUSPEND do and why is it important?
Q6	What Snowflake edition do you need for multi-cluster warehouses?
Q7	If 100 users run concurrent queries and queries are slow, would you scale UP or scale OUT? Why?
Q8	What is the Cloud Services layer and what does it do?
Q9	How does Snowflake store data in the storage layer?
Q10	What is the difference between STANDARD and ECONOMY scaling policies?

SESSION 3 — Data Types, Objects & Table Types

3.1 Snowflake Data Types

Snowflake supports all standard SQL data types plus special types for semi-structured data. Understanding data types is essential for designing good tables.

Numeric Data Types

Type	Description	Example
NUMBER(p,s)	Exact numeric with precision p and scale s. Also: DECIMAL, NUMERIC	NUMBER(10,2) → 12345678.90
INT / INTEGER	Whole numbers (38 digits max)	employee_id INT
FLOAT / DOUBLE	Approximate floating point (IEEE 754)	salary FLOAT
REAL	Single precision float	REAL

String / Text Data Types

Type	Max Length	Notes
VARCHAR(n)	16 MB	Most common string type. Default if n not specified: 16 MB
CHAR(n)	16 MB	Fixed length (padded with spaces)
STRING	16 MB	Alias for VARCHAR
TEXT	16 MB	Alias for VARCHAR

TIP Unlike other databases, VARCHAR in Snowflake does NOT waste space on shorter strings. A VARCHAR(1000) storing 'Hi' only uses 2 characters of storage. No need to be precise about sizes.

Date/Time Data Types

Type	Format	Example
DATE	YYYY-MM-DD	2024-01-15
TIME	HH:MI:SS.nnnnnnnnn	14:30:00.000000000
DATETIME / TIMESTAMP_NTZ	YYYY-MM-DD HH:MI:SS (no timezone)	2024-01-15 14:30:00
TIMESTAMP_LTZ	Timestamp with LOCAL timezone	Adjusts to session timezone
TIMESTAMP_TZ	Timestamp WITH stored timezone offset	2024-01-15 14:30:00 +05:30

Boolean & Binary

Type	Values	Notes

BOOLEAN	TRUE / FALSE / NULL	Supports NULL unlike some databases
BINARY / VARBINARY	Hex byte sequences	Stores raw binary data up to 8 MB

Semi-Structured Data Types (Snowflake's Superpower)

This is where Snowflake shines! You can store JSON, XML, Avro, Parquet, and ORC natively.

Type	Use Case	Max Size
VARIANT	Stores ANY semi-structured data (JSON, XML, etc.) in a flexible column	16 MB
OBJECT	Key-value pairs (like a JSON object). Keys must be strings.	16 MB
ARRAY	Ordered list of values (like a JSON array)	16 MB

```
-- Example: Storing JSON in a VARIANT column
CREATE TABLE customer_events (
  event_id   NUMBER,
  event_time TIMESTAMP_NTZ,
  event_data VARIANT    -- stores entire JSON blob
);

-- Insert JSON data:
INSERT INTO customer_events VALUES (
  1,
  CURRENT_TIMESTAMP,
  PARSE_JSON('{"user":"Alice","action":"purchase","amount":99.99}')
);

-- Query inside the JSON (dot notation):
SELECT event_data:user::STRING as username,
       event_data:amount::FLOAT as amount
FROM customer_events;
```

3.2 Snowflake Database Objects Hierarchy

Snowflake organizes everything in a clear hierarchy. You must understand this to navigate and write SQL correctly.

```
ORGANIZATION
  └── ACCOUNT  (your Snowflake account, e.g., xy12345.us-east-1)
        └── DATABASE  (e.g., SALES_DB)
              └── SCHEMA  (e.g., PUBLIC, RAW, ANALYTICS)
                    ├── TABLE
                    ├── VIEW
                    ├── STAGE
                    ├── SEQUENCE
                    ├── STORED PROCEDURE
```

```
                    ├── UDF (User Defined Function)
                    ├── STREAM
                    └── TASK
```

Fully qualified name format:

```
DATABASE_NAME.SCHEMA_NAME.OBJECT_NAME
-- Example:
SALES_DB.PUBLIC.CUSTOMERS       -- table
SALES_DB.ANALYTICS.V_REVENUE    -- view
```

Lab 2: Create Database Objects

```
-- Step 1: Create a Database
CREATE DATABASE training_db;

-- Step 2: Create Schemas
CREATE SCHEMA training_db.raw;        -- raw ingested data
CREATE SCHEMA training_db.analytics;  -- transformed data

-- Step 3: Use a schema
USE DATABASE training_db;
USE SCHEMA raw;

-- Step 4: Create a table
CREATE TABLE employees (
  emp_id     NUMBER       PRIMARY KEY,
  first_name VARCHAR(50),
  last_name  VARCHAR(50),
  hire_date  DATE,
  salary     NUMBER(10,2),
  is_active  BOOLEAN      DEFAULT TRUE,
  metadata   VARIANT
);

-- Step 5: Insert sample data
INSERT INTO employees (emp_id, first_name, last_name, hire_date, salary, metadata) SELECT emp_id, first_name, last_name, TO_DATE(hire_date), salary, PARSE_JSON(metadata) FROM VALUES (1, 'Priya', 'Sharma', '2020-01-15', 75000.00, '{"dept":"Engineering","skills":["SQL","Python"]}'), (2, 'Rahul', 'Verma', '2019-06-01', 85000.00, '{"dept":"Data Science","skills":["ML","R"]}'), (3, 'Ananya', 'Patel', '2021-03-20', 65000.00, '{"dept":"Engineering","skills":["Java","SQL"]}') AS vals(emp_id, first_name, last_name, hire_date, salary, metadata);

-- Step 6: Query
SELECT * FROM employees;
SELECT emp_id, first_name, metadata:dept::STRING as department FROM employees;
```

3.3 Table Types in Snowflake

Snowflake has 4 types of tables, each with different persistence and data retention behaviors. Choosing the right type saves money.

1. Permanent Tables (Default)

- Default table type. Data persists indefinitely until explicitly dropped.
- Supports: Time Travel (up to 90 days on Enterprise, 1 day on Standard/Free)
- Supports: Fail-safe (7 days extra protection by Snowflake, non-recoverable by users)
- Storage cost: charged for data + Time Travel data + Fail-safe data

```
-- Permanent table (default):
CREATE TABLE orders (order_id INT, amount FLOAT, order_date DATE);
-- Same as:
CREATE PERMANENT TABLE orders (order_id INT, amount FLOAT, order_date DATE);
```

2. Transient Tables

- Data persists across sessions (unlike Temporary)
- NO Fail-safe period (saves storage cost vs Permanent)
- Time Travel: 0 or 1 day only
- Best for: staging tables, intermediate ETL data, data you can easily recreate

```
CREATE TRANSIENT TABLE staging_orders (
  order_id   INT,
  amount     FLOAT,
  load_time  TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP
);
-- Saves ~14% on storage costs (no 7-day fail-safe)
```

3. Temporary Tables

- Exist ONLY for the current session. Automatically dropped when session ends.
- Not visible to other sessions (session-scoped)
- No Fail-safe
- Best for: intermediate calculations, debugging, personal scratch work

```
CREATE TEMPORARY TABLE temp_calc AS
SELECT emp_id, salary * 1.10 as new_salary FROM employees;

-- This table disappears when your session ends!
SELECT * FROM temp_calc;
```

4. External Tables

- Data lives in an external cloud storage location (S3 bucket, Azure Blob, GCS)
- Snowflake does NOT own the data — it just reads it
- Data is read-only — you cannot INSERT/UPDATE/DELETE
- No Time Travel, No Fail-safe
- Useful for: querying data without loading it, data lake integration

```
-- External table points to files in S3:
CREATE EXTERNAL TABLE ext_sales
  USING TEMPLATE (
    SELECT ARRAY_AGG(OBJECT_CONSTRUCT(*)) FROM TABLE(
      INFER_SCHEMA(LOCATION=>'@my_s3_stage', FILE_FORMAT=>'my_csv_format'))
```

```
)
LOCATION = @my_s3_stage
FILE_FORMAT = (TYPE = 'CSV' SKIP_HEADER = 1);
```

Feature	Permanent	Transient	Temporary	External
Persistence	Forever	Until dropped	Session only	Data in S3/GCS/Azure
Time Travel	0-90 days	0-1 day	0-1 day	None
Fail-safe	7 days	None	None	None
DML Operations	Yes	Yes	Yes	Read-only
Storage Cost	Highest	Lower	Low	Very low (metadata only)
Best For	Production data	ETL staging	Scratch calculations	Data lake queries

Questions — Session 3

Q1	What is the maximum size of a VARIANT column in Snowflake?
Q2	What is the difference between TIMESTAMP_NTZ, TIMESTAMP_LTZ and TIMESTAMP_TZ?
Q3	Why would you use a TRANSIENT table instead of a PERMANENT table?
Q4	What happens to a TEMPORARY table when your session ends?
Q5	Can you INSERT data into an External Table? Why or why not?
Q6	What is Fail-safe and which table types support it?
Q7	Write SQL to create a table with columns: id (integer), name (text), score (decimal 5,2), created_at (timestamp without timezone), extra_info (JSON-compatible).
Q8	What is the full qualified object name format in Snowflake?

SESSION 4 — Cloning, Time Travel & Drop/Undrop

4.1 Zero Copy Cloning

Zero Copy Cloning is one of Snowflake's most powerful and unique features. It lets you create an instant copy of a database, schema, or table WITHOUT physically copying the data.

How does it work?

- Snowflake uses pointers (references) to the existing micro-partitions (data files)
- The clone shares the same underlying storage as the original — no data is duplicated
- The clone is fully independent — changes to the clone do NOT affect the original and vice versa
- When you modify data in the clone, NEW micro-partitions are created only for the changed data (copy-on-write)

Real-world analogy:

Imagine a shared Google Doc. When you share the doc, both people see the same content instantly. If Person A edits it, their changes create a new version for them — the original is untouched. That is copy-on-write.

What can you clone?

- Tables
- Schemas (clones the schema AND all tables inside)
- Databases (clones everything inside)

```
-- Clone a TABLE:
CREATE TABLE employees_backup
  CLONE employees;

-- Clone a TABLE at a specific point in time (time-travel clone):
CREATE TABLE employees_yesterday
  CLONE employees
  AT (TIMESTAMP => DATEADD(DAY, -1, CURRENT_TIMESTAMP));

-- Clone a SCHEMA (copies all tables in it):
CREATE SCHEMA analytics_backup
  CLONE analytics;

-- Clone an entire DATABASE:
CREATE DATABASE training_db_dev
  CLONE training_db;

-- Verify the clone:
SELECT COUNT(*) FROM employees;          -- original
SELECT COUNT(*) FROM employees_backup;   -- clone (same count)
```

TIP Common use case: Clone your production database to create a development environment. Your dev team gets full data instantly — no ETL, no waiting, no extra storage cost (until they start modifying data).

NOTE Zero Copy Cloning is available on ALL editions including the free trial.

4.2 Data Retention & Time Travel

Time Travel allows you to query or restore data from the PAST. This is a safety net against accidental deletes and a powerful analytical tool.

Data Retention Period

- Standard Edition / Free Trial: Maximum 1 day (86400 seconds)
- Enterprise Edition: Maximum 90 days
- Default: 1 day for all editions

```
-- Set data retention when creating table:
CREATE TABLE orders (order_id INT, amount FLOAT)
  DATA_RETENTION_TIME_IN_DAYS = 1;  -- free tier max

-- Change retention on existing table:
ALTER TABLE orders SET DATA_RETENTION_TIME_IN_DAYS = 1;

-- Check current retention setting:
SHOW TABLES LIKE 'orders';
```

FREE TIER Free tier supports maximum 1 day Time Travel. The 90-day Time Travel is an Enterprise feature.

3 Ways to Query Historical Data with Time Travel

Method 1: AT TIMESTAMP — go to a specific date/time

```
-- See what the employees table looked like at a specific timestamp:
SELECT * FROM employees
  AT (TIMESTAMP => '2024-01-15 10:00:00'::TIMESTAMP_NTZ);

-- Go back 1 hour:
SELECT * FROM employees
  AT (TIMESTAMP => DATEADD(HOUR, -1, CURRENT_TIMESTAMP));

-- Go back 30 minutes:
SELECT * FROM employees
  AT (TIMESTAMP => DATEADD(MINUTE, -30, CURRENT_TIMESTAMP));
```

Method 2: BEFORE STATEMENT — reference a specific query that ran

```
-- First, find the Query ID of the statement that made changes:
-- (Get from query history in Snowsight, or:
SELECT LAST_QUERY_ID();  -- after running a statement

-- Then query the state BEFORE that specific statement ran:
SELECT * FROM employees
  BEFORE (STATEMENT => '01ab2cd3-0000-1234-ab12-000000000001');

-- Real scenario: You accidentally deleted all employees. Get the ID of
DELETE statement.
-- Then restore:
INSERT INTO employees
```

```
  SELECT * FROM employees
    BEFORE (STATEMENT => '<delete_query_id>');
```

Method 3: AT OFFSET — go back a number of seconds

```
-- Go back 3600 seconds (1 hour):
SELECT * FROM employees
  AT (OFFSET => -3600);

-- Go back 300 seconds (5 minutes):
SELECT * FROM employees
  AT (OFFSET => -300);

-- Combine with CLONE to restore a table:
-- Step 1: Clone the table from 1 hour ago
CREATE TABLE employees_restored
  CLONE employees AT (OFFSET => -3600);

-- Step 2: Drop the corrupted table
DROP TABLE employees;

-- Step 3: Rename the restored clone
ALTER TABLE employees_restored RENAME TO employees;
```

4.3 DROP and UNDROP

DROP TABLE / SCHEMA / DATABASE

When you DROP a table in Snowflake, it is NOT immediately deleted. It enters a dropped state and can be recovered during the retention period.

```
-- Drop a table:
DROP TABLE employees;

-- The table appears gone from SHOW TABLES
-- But internally Snowflake keeps it for retention period

-- Drop a schema (drops all objects inside):
DROP SCHEMA analytics;

-- Drop a database (drops everything inside):
DROP DATABASE training_db;
```

UNDROP — Recover a Dropped Object

UNDROP brings back a dropped table, schema, or database within the retention period. This is your undo button.

```
-- Recover a dropped table:
UNDROP TABLE employees;

-- Recover a dropped schema (and all tables in it):
```

```
UNDROP SCHEMA analytics;

-- Recover a dropped database:
UNDROP DATABASE training_db;

-- IMPORTANT: If a table with the same name was created AFTER dropping,
-- you must first rename or drop the new table before UNDROP.
```

Complete Lab — Time Travel Scenario

> **NOTE** This lab works on FREE tier. Demonstrates all 3 Time Travel methods.

```
-- SETUP: Create test table and insert data
USE DATABASE training_db;
USE SCHEMA raw;

CREATE OR REPLACE TABLE products (
  product_id   INT,
  product_name VARCHAR(100),
  price        NUMBER(8,2)
);

INSERT INTO products VALUES
  (1, 'Laptop',  75000),
  (2, 'Mouse',   1500),
  (3, 'Keyboard',2500);

-- *** SAVE THIS TIMESTAMP: ***
SELECT CURRENT_TIMESTAMP;  -- copy this value!

-- Simulate accident: delete all rows
DELETE FROM products;
SELECT COUNT(*) FROM products;  -- should be 0

-- ========================================
-- RECOVERY METHOD 1: AT TIMESTAMP
-- ========================================
-- Replace <TIMESTAMP> with the value you saved above:
SELECT * FROM products
  AT (TIMESTAMP => '<your_timestamp>'::TIMESTAMP_NTZ);

-- Restore from timestamp:
INSERT INTO products
  SELECT * FROM products
    AT (TIMESTAMP => '<your_timestamp>'::TIMESTAMP_NTZ);

SELECT * FROM products;  -- should show 3 rows again
```

```
-- ========================================
-- RECOVERY METHOD 2: BEFORE STATEMENT
-- ========================================
-- Delete again to practice method 2:
DELETE FROM products;

-- Get the query ID of the DELETE statement:
SELECT LAST_QUERY_ID();  -- copy this ID

-- Recover using BEFORE STATEMENT:
INSERT INTO products
  SELECT * FROM products
    BEFORE (STATEMENT => '<delete_query_id>');

-- ========================================
-- RECOVERY METHOD 3: OFFSET
-- ========================================
DELETE FROM products;

INSERT INTO products
  SELECT * FROM products
    AT (OFFSET => -300);  -- go back 5 minutes

-- UNDROP demo:
DROP TABLE products;
SELECT * FROM products;   -- Error: table does not exist
UNDROP TABLE products;
SELECT * FROM products;   -- 3 rows back!
```

Questions — Session 4

Q1	What does 'Zero Copy' mean in Snowflake cloning? How does it work internally?
Q2	Can you clone a database? What gets cloned along with it?
Q3	What is the maximum Time Travel period on Standard Edition vs Enterprise Edition?
Q4	Name and explain all 3 ways to use Time Travel in Snowflake.
Q5	What happens when you DROP a table in Snowflake — is the data immediately deleted?
Q6	If you dropped a table and created a new table with the same name, can you UNDROP the old one?
Q7	What is the difference between Time Travel and Fail-safe?
Q8	A developer accidentally ran DELETE FROM orders WHERE 1=1. How would you recover the data?
Q9	What is copy-on-write behavior in the context of cloning?
Q10	What is DATA_RETENTION_TIME_IN_DAYS and where can you set it?

SESSION 5 — Views, Materialized Views & Stages

5.1 Views

A View is a named query stored in the database. When you query a view, Snowflake runs the underlying SQL and returns the result. Views do NOT store data — they are virtual.

```
-- Create a standard view:
CREATE OR REPLACE VIEW v_active_employees AS
SELECT
  emp_id,
  first_name || ' ' || last_name AS full_name,
  salary,
  metadata:dept::STRING AS department
FROM employees
WHERE is_active = TRUE;

-- Query the view just like a table:
SELECT * FROM v_active_employees WHERE department = 'Engineering';

-- Show all views in current schema:
SHOW VIEWS;

-- See the SQL behind a view:
SELECT GET_DDL('VIEW', 'v_active_employees');
```

Pros: simple to create, always shows latest data, no extra storage cost.

Cons: runs the underlying query every time (no caching), can be slow on complex queries over large datasets.

Secure Views

A Secure View hides the view definition from users who do not own it. The underlying SQL is encrypted and not visible.

```
-- Create a secure view (hides the SQL definition):
CREATE OR REPLACE SECURE VIEW v_salary_report AS
SELECT emp_id, full_name, salary
FROM v_active_employees;

-- Only the owner can see the definition
-- Other users see data but cannot see the SQL
SHOW VIEWS; -- security_type column shows 'SECURE'
```

NOTE Secure views are important for data security. Use them when you do not want to expose business logic or filter conditions to end users.

5.2 Materialized Views

A Materialized View (MV) pre-computes and STORES the query result physically. When you query an MV, Snowflake returns the stored result (very fast) instead of re-running the query.

Key differences from regular views:

- Data is physically stored (takes up storage)
- Snowflake automatically refreshes the MV when the base table changes (background maintenance)
- Much faster for expensive aggregate queries
- Requires Enterprise Edition for full functionality

```
-- Create a Materialized View:
CREATE MATERIALIZED VIEW mv_dept_salary AS
SELECT
  metadata:dept::STRING AS department,
  COUNT(*) AS employee_count,
  AVG(salary) AS avg_salary,
  SUM(salary) AS total_salary
FROM employees
GROUP BY metadata:dept::STRING;

-- Query it — very fast even on huge tables:
SELECT * FROM mv_dept_salary WHERE avg_salary > 70000;

-- View MV details:
SHOW MATERIALIZED VIEWS;
```

 FREE TIER Materialized Views require Enterprise Edition. On Standard/Free, you can create them but maintenance credits may apply differently. Basic MV creation works on free trial — test in your account.

Feature	Regular View	Materialized View
Data storage	None (virtual)	Yes (physical storage)
Query speed	Runs query each time	Reads pre-computed result (fast)
Freshness	Always current	Auto-refreshed by Snowflake
Cost	Compute cost per query	Storage + maintenance cost
Best for	Simple filters/logic	Expensive aggregations
Edition required	All editions	Enterprise recommended

5.3 Stages — The Loading/Unloading Gateway

A Stage is a location where data files are stored BEFORE loading into Snowflake, or AFTER exporting from Snowflake. Think of it as a staging area (like a loading dock).

Snowflake supports 2 main types of stages:

Internal Stages (Snowflake manages the storage)

There are 3 kinds of internal stages:

1. User Stage — personal stage for each user

```
-- Each user has their own stage: @~
-- Upload a file to your user stage:
-- (via SnowSQL command line):
```

```
PUT file:///local/path/employees.csv @~;

-- List files in your user stage:
LIST @~;

-- Load from user stage:
COPY INTO employees FROM @~/employees.csv
  FILE_FORMAT = (TYPE = 'CSV' FIELD_OPTIONALLY_ENCLOSED_BY = '"' SKIP_HEADER
= 1);
```

2. Table Stage — dedicated stage for a specific table

```
-- Every table has its own stage: @%tablename
-- Upload to table stage (SnowSQL):
PUT file:///local/orders.csv @%orders;

-- List files in table stage:
LIST @%orders;

-- Load from table stage:
COPY INTO orders FROM @%orders
  FILE_FORMAT = (TYPE = 'CSV' SKIP_HEADER = 1);
```

3. Named Stage — reusable stage you create

```
-- Create a named internal stage:
CREATE OR REPLACE STAGE my_internal_stage
  COMMENT = 'General purpose internal stage';

-- Upload (SnowSQL):
PUT file:///local/data/*.csv @my_internal_stage;

-- List files:
LIST @my_internal_stage;

-- Preview files without loading:
SELECT $1, $2, $3 FROM @my_internal_stage/employees.csv
  (FILE_FORMAT => (TYPE='CSV' SKIP_HEADER=1));

-- Delete files from stage:
REMOVE @my_internal_stage/employees.csv;
```

External Stages (points to cloud storage you own)

An external stage is a named reference to a cloud bucket (AWS S3, Azure Blob, GCS) that YOU own.

```
-- Create an external stage pointing to S3:
CREATE OR REPLACE STAGE my_s3_stage
  URL = 's3://my-company-bucket/data/'
  CREDENTIALS = (AWS_KEY_ID = 'AKIA...' AWS_SECRET_KEY = 'xxx...')
  FILE_FORMAT = (TYPE = 'CSV' FIELD_OPTIONALLY_ENCLOSED_BY = '"' SKIP_HEADER
= 1);
```

```
-- Better practice: use STORAGE INTEGRATION (no credentials in SQL):
CREATE OR REPLACE STAGE my_secure_s3_stage
  URL = 's3://my-company-bucket/data/'
  STORAGE_INTEGRATION = s3_int
  FILE_FORMAT = (TYPE = 'PARQUET');

-- List files in external stage:
LIST @my_s3_stage;

-- Query files in external stage directly:
SELECT $1, $2 FROM @my_s3_stage;
```

 NOTE Storage Integration is the recommended way to connect to cloud storage — it uses IAM roles instead of embedding credentials in SQL, which is more secure.

5.4 Loading Data — COPY INTO

The COPY INTO command is the primary way to bulk-load data into Snowflake tables. It reads files from a stage and inserts them into a table.

```
-- Basic COPY INTO:
COPY INTO employees
FROM @my_internal_stage/employees.csv
FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1 FIELD_OPTIONALLY_ENCLOSED_BY='"');

-- Load multiple files (pattern matching):
COPY INTO orders
FROM @my_s3_stage
PATTERN = '.*orders_2024.*\.csv'
FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1);

-- Load JSON files:
COPY INTO events
FROM @my_stage/events/
FILE_FORMAT = (TYPE='JSON' STRIP_OUTER_ARRAY=TRUE);

-- Check load history:
SELECT * FROM INFORMATION_SCHEMA.LOAD_HISTORY
WHERE TABLE_NAME = 'EMPLOYEES'
ORDER BY LAST_LOAD_TIME DESC;
```

VALIDATION_MODE — Test Before Loading

Before loading real data, you can validate files to catch errors:

```
-- RETURN_ERRORS: show all errors without loading any data
COPY INTO employees FROM @my_stage/employees.csv
FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1)
VALIDATION_MODE = 'RETURN_ERRORS';
```

```
-- RETURN_n_ROWS: load a sample of n rows to preview
COPY INTO employees FROM @my_stage/employees.csv
FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1)
VALIDATION_MODE = 'RETURN_10_ROWS';

-- RETURN_ALL_ERRORS: return ALL rows with errors (not just first batch)
COPY INTO employees FROM @my_stage/employees.csv
FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1)
VALIDATION_MODE = 'RETURN_ALL_ERRORS';
```

 TIP Always run VALIDATION_MODE = 'RETURN_ERRORS' first when loading new data. It catches data quality issues before they get into your tables.

Unloading Data from Snowflake

You can also export data FROM Snowflake back to a stage using COPY INTO with a reversed direction:

```
-- Export to internal stage:
COPY INTO @my_internal_stage/export/employees_
FROM employees
FILE_FORMAT = (TYPE='CSV' HEADER=TRUE)
SINGLE = FALSE         -- creates multiple files for parallel write
MAX_FILE_SIZE = 52428800;  -- 50MB per file

-- Export to S3 external stage:
COPY INTO @my_s3_stage/export/
FROM (SELECT emp_id, first_name, salary FROM employees WHERE salary > 70000)
FILE_FORMAT = (TYPE='CSV' HEADER=TRUE);
```

Lab 5: Stages and Loading

 NOTE This lab works on FREE tier. We use internal stages (no external cloud account needed).

```
-- Step 1: Create a named internal stage
CREATE OR REPLACE STAGE training_stage
  COMMENT = 'Stage for training labs';

-- Step 2: Create a file format
CREATE OR REPLACE FILE FORMAT csv_format
  TYPE = 'CSV'
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  SKIP_HEADER = 1
  DATE_FORMAT = 'YYYY-MM-DD'
  TIMESTAMP_FORMAT = 'YYYY-MM-DD HH24:MI:SS'
  NULL_IF = ('NULL', 'null', '');

-- Step 3: Upload a file using SnowSQL (from your laptop terminal):
-- snowsql -a <account> -u <user>
-- PUT file:///c:/data/employees.csv @training_stage;  (Windows)
-- PUT file:///home/user/employees.csv @training_stage;  (Mac/Linux)
```

```
-- Step 4: List files in stage:
LIST @training_stage;

-- Step 5: Preview file contents:
SELECT $1, $2, $3 FROM @training_stage (FILE_FORMAT => csv_format) LIMIT 5;

-- Step 6: Validate before loading:
COPY INTO employees FROM @training_stage
FILE_FORMAT = (FORMAT_NAME = 'csv_format')
VALIDATION_MODE = 'RETURN_ERRORS';

-- Step 7: Load data:
COPY INTO employees FROM @training_stage
FILE_FORMAT = (FORMAT_NAME = 'csv_format');

-- Step 8: Verify:
SELECT COUNT(*) FROM employees;

-- Step 9: Export data back out:
COPY INTO @training_stage/export/employees_out
FROM employees
FILE_FORMAT = (TYPE='CSV' HEADER=TRUE);

LIST @training_stage/export/;
```

Questions — Session 5

Q1	What is the difference between a regular view and a materialized view?
Q2	What is a Secure View and when would you use it?
Q3	Name the 3 types of internal stages in Snowflake.
Q4	What is the difference between an internal stage and an external stage?
Q5	What command do you use to upload files to an internal stage?
Q6	What does COPY INTO VALIDATION_MODE = 'RETURN_ERRORS' do?
Q7	What is a FILE FORMAT in Snowflake and why is it useful?
Q8	How would you export data from a Snowflake table to a CSV file?
Q9	What is the purpose of STORAGE INTEGRATION when creating external stages?
Q10	Can you query data in a stage without loading it into a table? How?

SESSION 6 — Comprehensive Lab + Day 1 Summary

Comprehensive Lab — End-to-End Scenario

> **NOTE** Full lab works on FREE tier Snowflake. Covers ALL concepts.

Scenario: You are a Data Engineer at RetailMart. Set up the complete data infrastructure for the sales team.

```
-- ================================================
-- RETAILMART DATA PLATFORM SETUP
-- ================================================

-- PART 1: Infrastructure Setup
-- ================================================
CREATE DATABASE retailmart_db;
CREATE SCHEMA retailmart_db.raw;
CREATE SCHEMA retailmart_db.analytics;

CREATE WAREHOUSE retailmart_wh
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 120
  AUTO_RESUME = TRUE;

USE DATABASE retailmart_db;
USE SCHEMA raw;
USE WAREHOUSE retailmart_wh;

-- PART 2: Create Tables
-- ================================================
-- Permanent table for customers (keep forever)
CREATE TABLE customers (
  customer_id   NUMBER PRIMARY KEY,
  customer_name VARCHAR(100),
  email         VARCHAR(200),
  city          VARCHAR(100),
  signup_date   DATE
) DATA_RETENTION_TIME_IN_DAYS = 1;

-- Permanent table for orders
CREATE TABLE orders (
  order_id    NUMBER PRIMARY KEY,
  customer_id NUMBER,
  order_date  DATE,
  amount      NUMBER(10,2),
  status      VARCHAR(20)
) DATA_RETENTION_TIME_IN_DAYS = 1;

-- Transient table for staging (no fail-safe needed)
```

```
CREATE TRANSIENT TABLE orders_staging (
  order_id    NUMBER,
  customer_id NUMBER,
  order_date  VARCHAR(20),  -- raw string from CSV
  amount      VARCHAR(20),  -- raw string
  status      VARCHAR(20)
);

-- PART 3: Insert Sample Data
-- ================================================
INSERT INTO customers VALUES
  (1, 'Amit Shah',   'amit@email.com',  'Mumbai',    '2022-01-10'),
  (2, 'Pooja Singh',  'pooja@email.com', 'Delhi',     '2022-03-15'),
  (3, 'Rohan Mehta',  'rohan@email.com', 'Bangalore', '2023-06-01'),
  (4, 'Sneha Rao',    'sneha@email.com', 'Chennai',   '2023-09-20');

INSERT INTO orders VALUES
  (101, 1, '2024-01-05', 2500.00, 'COMPLETED'),
  (102, 2, '2024-01-10', 4200.00, 'COMPLETED'),
  (103, 1, '2024-02-01', 1800.00, 'PENDING'),
  (104, 3, '2024-02-15', 9500.00, 'COMPLETED'),
  (105, 4, '2024-03-01', 3300.00, 'CANCELLED');

-- PART 4: Analytics Views
-- ================================================
USE SCHEMA analytics;

CREATE OR REPLACE VIEW v_customer_orders AS
SELECT
  c.customer_id,
  c.customer_name,
  c.city,
  COUNT(o.order_id)    AS total_orders,
  SUM(o.amount)        AS total_spent,
  MAX(o.order_date)    AS last_order_date
FROM retailmart_db.raw.customers c
LEFT JOIN retailmart_db.raw.orders o
  ON c.customer_id = o.customer_id
WHERE o.status = 'COMPLETED'
GROUP BY c.customer_id, c.customer_name, c.city;

SELECT * FROM v_customer_orders ORDER BY total_spent DESC;

-- PART 5: Zero Copy Clone (Dev Environment)
-- ================================================
CREATE DATABASE retailmart_dev CLONE retailmart_db;
```

```
-- Your dev team can now experiment freely:
USE DATABASE retailmart_dev;
USE SCHEMA raw;
DELETE FROM orders;  -- dev team deletes in dev DB
SELECT COUNT(*) FROM orders;  -- 0 in dev

-- Original is untouched:
SELECT COUNT(*) FROM retailmart_db.raw.orders;  -- still 5 rows!

-- PART 6: Time Travel Demo
-- ================================================
USE DATABASE retailmart_db;
USE SCHEMA raw;

-- Note current time
SELECT CURRENT_TIMESTAMP AS before_delete;

-- Accidental deletion:
DELETE FROM orders WHERE status = 'COMPLETED';
SELECT COUNT(*) FROM orders;  -- should show only 2 (PENDING + CANCELLED)

-- Recover using OFFSET:
INSERT INTO orders
  SELECT * FROM orders AT (OFFSET => -120);  -- go back 2 minutes
SELECT COUNT(*) FROM orders;  -- all 5 back

-- PART 7: Stage and Load
-- ================================================
CREATE STAGE retailmart_stage;

-- Export orders to stage:
COPY INTO @retailmart_stage/orders_backup
FROM orders
FILE_FORMAT = (TYPE='CSV' HEADER=TRUE);

LIST @retailmart_stage;
```

Summary — Key Concepts Recap

Concept	Key Takeaway
Shared Disk	All nodes share one storage — storage bottleneck
Shared Nothing	Each node has own disk — scaling storage+compute together
Snowflake Architecture	3 layers: Storage (S3 etc.), Compute (Virtual WH), Cloud Services (brain)
Editions	Standard (free), Enterprise (multi-cluster, 90d TT), Business Critical, Virtual Private

Virtual Warehouses	Named compute clusters. Scale up = bigger size. Scale out = more clusters.
Auto Suspend	Always set it! Prevents surprise bills.
Table Types	Permanent (full), Transient (no fail-safe), Temporary (session), External (read-only)
Zero Copy Clone	Instant copy, shared storage, copy-on-write. For dev/test environments.
Time Travel	3 methods: AT TIMESTAMP, BEFORE STATEMENT, AT OFFSET. Max 1 day free tier.
UNDROP	Recover dropped tables/schemas/databases within retention period.
Views	Virtual (no storage). Materialized = physical copy, auto-refreshed.
Stages	Internal (Snowflake manages) or External (your cloud bucket). Loading dock.
COPY INTO	Bulk load from stage to table. VALIDATION_MODE to test first.

What's Coming ?

- Bulk Loading, Continuous Loading, Snowpipe
- Streams, Tasks, DAGs, Stream+Task together
- UDFs, Table UDFs, Stored Procedures
- Semi-structured Data: Dot notation, FLATTEN, LATERAL FLATTEN
- Unstructured Data, Scoped URLs, Pre-signed URLs
- Micro-partitioning, Query Pruning, Clustering
- Query Profile, Data Spilling, Caching (Result, Metadata, Warehouse)
- Access Control: Roles, Privileges, Row-Level & Column-Level Security
- Dynamic Data Masking, External Tokenization
- Data Sharing: Provider/Consumer, Data Exchange, Replication
- Snowflake Scripting, Cost Optimization, Query Acceleration Service

---

